

Objects

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Object Literals

- A collection of related data and behavior as **key/value pairs**

```
const user = {  
  name: "Alice",  
  age: 30,  
  isAdmin: false,  
};  
  
console.log(user); // { name: 'Alice', age: 30, isAdmin: false }
```

- Values can be any type; pairs separated by commas
- An empty object is just `{}`

Property Access: Dot vs Bracket

```
const user = { name: "Alice", "favorite color": "blue" };

console.log(user.name); // "Alice" (dot notation)

const key = "name";
console.log(user[key]);           // "Alice" (key from a variable)
console.log(user["favorite color"]); // "blue" (space in the key)

console.log(user.email); // undefined (missing - no error)
```

Add, Update, Delete

```
const user = { name: "Alice" };

user.age = 30;           // add
user.name = "Alice S."; // update
delete user.age;         // delete

console.log(user); // { name: 'Alice S.' }
```

- Objects are **mutable**: `const` blocks reassignment, not mutation

```
const u = { name: "Alice" };
u.name = "Bob"; // OK - changing a property
// u = {};      // Error - reassigning the const variable
```

Nested Objects and Optional Chaining

```
const user = {  
  name: "Alice",  
  address: { city: "Seattle", zip: "98101" },  
};  
console.log(user.address.city); // "Seattle"
```

```
const bob = { name: "Bob" }; // no address  
console.log(bob.address.city); // TypeError  
console.log(bob.address?.city); // undefined (no error)
```

- Optional chaining `?.` short-circuits the **whole** rest of the chain

Defaults with ??

- `??` falls back only for `null` / `undefined`
- `||` falls back for **any** falsy value, including valid `0` and `""`

```
const settings = { fontSize: 0, title: "" };

settings.fontSize || 16; // 16  WRONG - 0 was a real choice
settings.fontSize ?? 16; // 0   right

settings.title || "Untitled"; // "Untitled"  WRONG
settings.title ?? "Untitled"; // ""          right

user.address?.city ?? "Unknown"; // the pair, together
```

Shorthand, Computed Keys, Methods

```
const name = "Alice", age = 30;  
const b = { name, age }; // shorthand properties  
  
const field = "email";  
const user = { name, [field]: "alice@example.com" }; // computed key
```

```
const calculator = {  
  value: 0,  
  add(n) { this.value += n; }, // method shorthand  
};  
calculator.add(5);  
console.log(calculator.value); // 5
```

Iterating Over an Object

```
const scores = { math: 90, science: 85, art: 95 };

Object.keys(scores);    // [ 'math', 'science', 'art' ]
Object.values(scores);  // [ 90, 85, 95 ]
Object.entries(scores); // [ ['math', 90], ... ]

for (const [subject, score] of Object.entries(scores)) {
  console.log(`${subject}: ${score}`);
}
```

Checking for a Property

```
const user = { name: "Alice", nickname: undefined };

Object.hasOwn(user, "name");    // true
Object.hasOwn(user, "email");   // false
Object.hasOwn(user, "nickname"); // true - it EXISTS, just undefined

user.email === undefined;      // true, but so is nickname
"name" in user;                 // true (also searches the prototype)
```

- `Object.hasOwn` is the modern replacement for `hasOwnProperty`

Copying, Merging, Freezing

```
const defaults = { theme: "light", fontSize: 14 };  
const custom = { fontSize: 18 };  
  
const settings = { ...defaults, ...custom }; // spread  
// { theme: 'light', fontSize: 18 } - later sources win
```

- Spread and `Object.assign` make **shallow** copies (nested is shared)
- `structuredClone(obj)` deep-copies plain data
- `Object.freeze(obj)` makes it read-only

Value vs Reference

Value vs Reference

Primitives copy the value. Objects copy the reference: two names, one object.

Primitives copy the VALUE

```
let a = 10;  
let b = a; // copies the number  
b = 20;  
// a is still 10
```

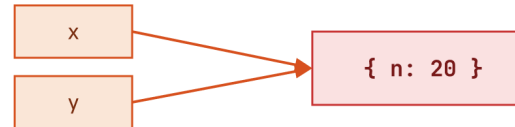
a → 10

b → 20

Two separate boxes. Changing one cannot touch the other. Same for string, boolean, number, null, undefined, symbol, bigint.

Objects copy the REFERENCE

```
const x = { n: 10 };  
const y = x; // copies the arrow, not the box  
y.n = 20;  
// x.n is now 20 too
```



One object, two names. Arrays and functions are objects too, so they behave the same way.

To get a real copy, spread it: `const z = { ...x }` or `const arr2 = [ ...arr ]`
But spread is SHALLOW: a nested object is still shared. Use `structuredClone(x)` for a deep copy.
Also why `const` does not mean immutable: it fixes the arrow, not the box it points at.

- Primitives copy the value; objects copy the reference
- Spread gives a **shallow** copy; a nested object is still shared